Greedy Algorithms

Fractional knapsack

Coin changing

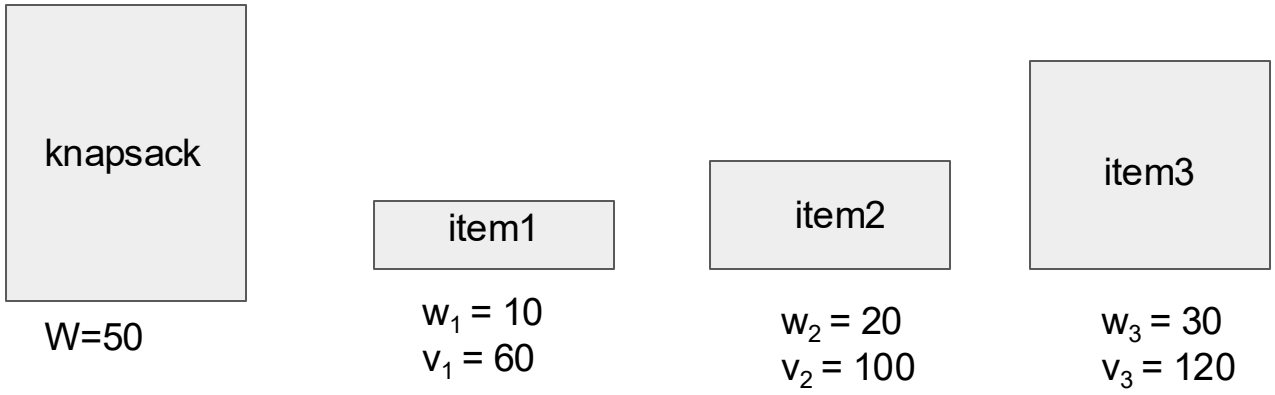
Readings

- Fractional knapsack
  - https://www.geeksforgeeks.org/fractional-knapsack-problem/
- Coin change:
- Chapter 16 - problem 16-1

Fractional knapsack

- **Input**:
  - n items with weight $w_1, w_2, \ldots, w_n$ and values $v_1, v_2, \ldots, v_n$
  - A knapsack with capacity W (total weight of items at most W)
  - The items are divisible: can put a fraction of an item into knapsack
- **Output**: maximize $\mathbf{p_1v_1+p_2v_2+ \ldots + p_nv_n}$ Constraint: $\mathbf{p_1w_1+p_2w_2+ \ldots + p_nw_n \leq W}$
  - Where $\mathbf{p_i}$ are the fraction of item i

$0 \leq p_i \leq 1$



Fractional knapsack

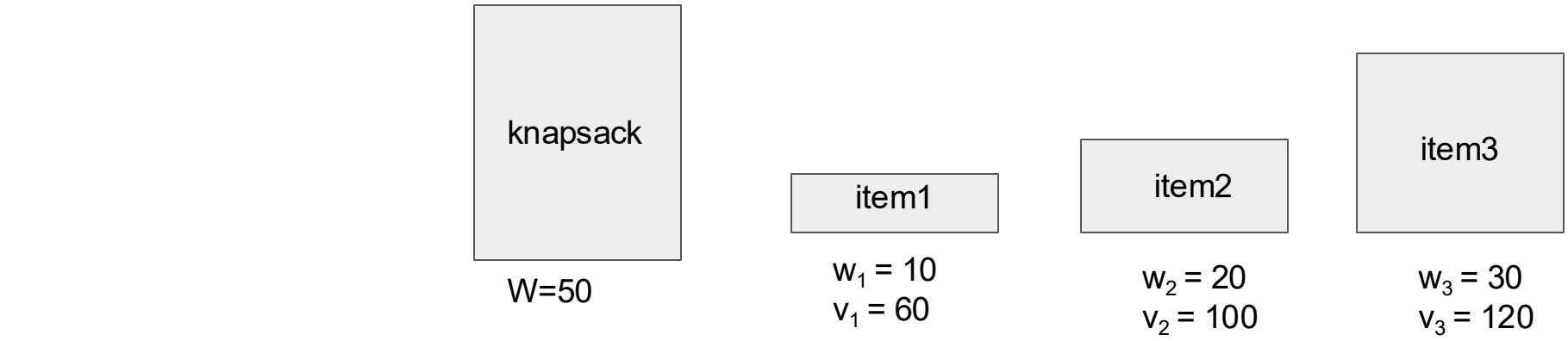
- **Greedy Algorithm 1:**
  - Sort items in decreasing order of value (take most valuable first)
  - 120 + 100 = 220 Not optimal

optimal value → 240

item 3 + item 2

weight = 30 + 20 = 50

value = 120 + 100 = 220



Fractional knapsack

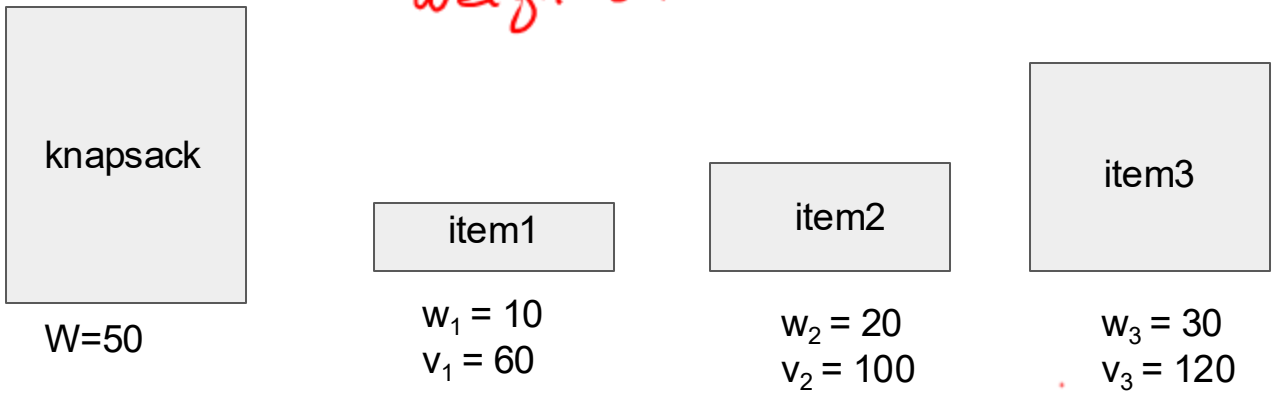
- **Greedy Algorithm 2:**
  - Sort item in increasing order of weight (take lightest item first)
  - 60+100+ (20/30) * 120 = 240
  - Looks optimal for this example.
  - How about this example: values = [20, 50, 100] amd weights=[10, 20, 100] and W=10

v_1 v_2 v_3 w_1 w_2 w_3

Algm 2 → take item 1 → weight = 10 value 20

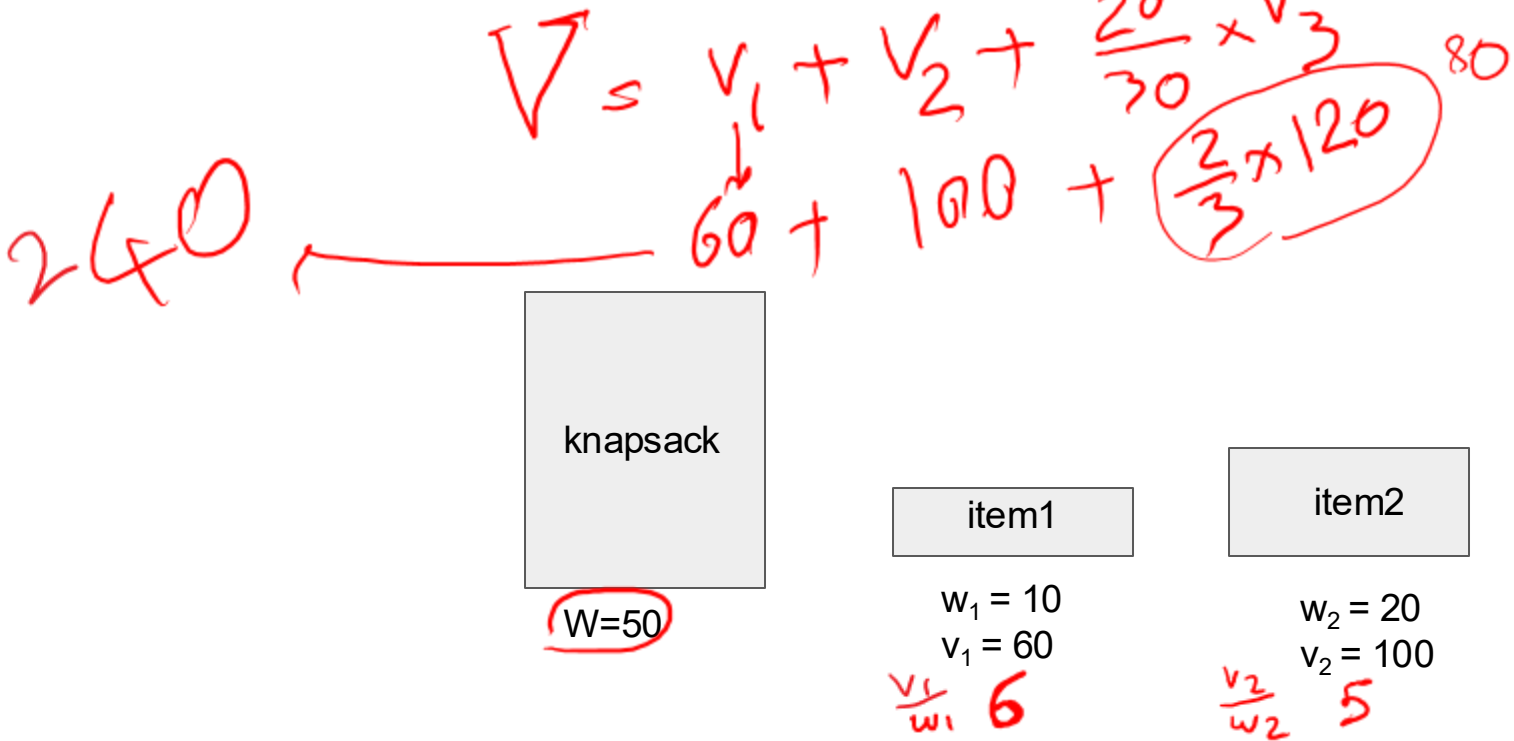
optimal solution → take hallf of second item →

weight = 10 value = 25



Fractional knapsack

- **Greedy Algorithm 3:**
  - Sort items in decreasing order of v_i/w_i
  - Take item with highest value per weight first



Fractional knapsack: greedy algorithm

- Pseudocode

- Runtime Analysis
  - O(n log n)

```
def Knapsack(W, v[1…n], w[1…n]):
    Sort items in decreasing order of v_i / w_i
    S = {}, V=0
    for i = 1 to n:
        if w_i < W:
                Put item i into S
                W = W-w_i
                V = V + V_i
        else:
            #put fraction of item i into S
            V = V + V_i  W/w_i
            break
     return V
```

Fractional knapsack: greedy algorithm 3: Proof of Optimality

- **The greedy algorithm gives an optimal solution to the fractional knapsack problem**
- **Proof:**
  - **X**: Greedy solution = $\{x_1, x_2, \dots, x_n\}$
  - **Y**: Optimal solution = $\{y_1, y_2, \dots, y_n\}$
  - We prove that **X** = **Y**
    - We prove that if the optimal solution is different from greedy algorithm, then there will be a solution that is better than optimal solution (has more value than optimal solution) which is a contradiction.
  - Suppose X is not equal to Y
  - Let j: the smallest integer j such that $x_j \neq y_j$

Fractional knapsack: greedy algorithm 3: Proof of Optimality

Greedy: $X = x_1 \; x_2 \; \dots \; x_{j-1} \; \mathbf{x_j} \; \dots \; \mathbf{x_k} \; \dots \; x_n$

Optimal: $Y = y_1 \; y_2 \; \dots \; y_{j-1} \; \mathbf{y_j} \; \dots \; \mathbf{y_k} \; \dots \; y_n$

$Y' = y_1 \; y_2 \; \dots \; y_{j-1} \; \mathbf{y'_j} \; \dots \; \mathbf{y'_k} \; \dots \; y_n$

x_i or y_i are the fraction of each item

- Assume the two solutions are identical up to item j-1. At item j, $\mathbf{x_j > y_j}$ because of the way algorithm works. The greedy algorithm takes as much as possible of the item with the **highest value/weight ratio**.
- Must exist $\mathbf{k > j}$ such that $\mathbf{y_k > 0}$ **and** $\mathbf{y_k > x_k}$. Why? Since both solutions must have the same total weight
- Exchange weight ε of item k for equal weight of item j in an optimal solution--> Y'
  - Add ε to item j and remove ε from item k
  - Total weight stays the same
  - Total value in Y increases since items are sorted by vaue/weight ratio $\frac{v_j}{w_j} \geq \frac{v_k}{w_k}$
- Contradiction: Y was optimal

Knapsack problem: Greedy vs dynamic programming

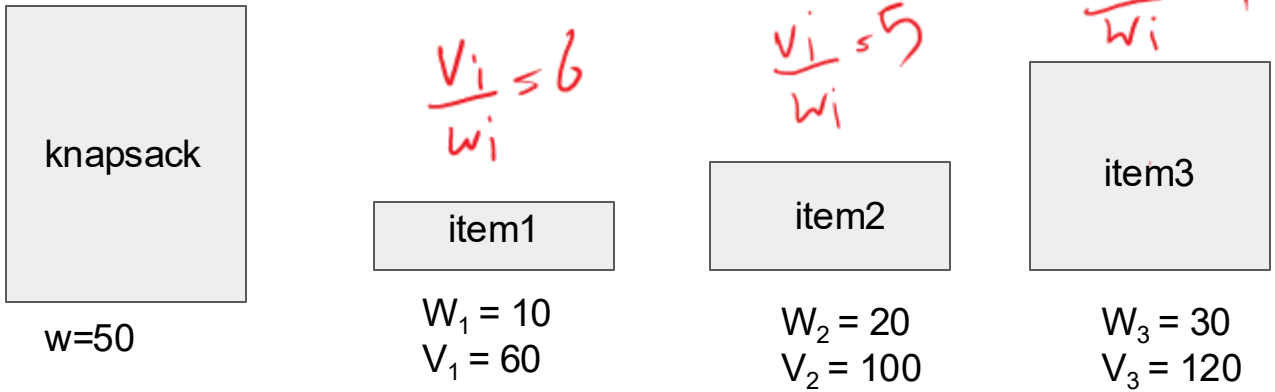
- Does the greedy solution works for 0-1 knapsack?
  - No

using greedy algm leads to this weight & value

W = 10 + 20

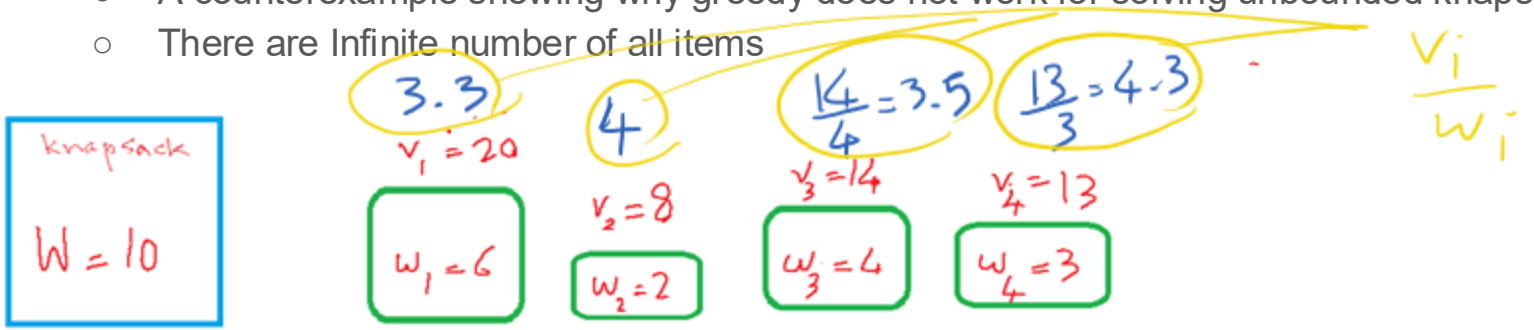
V = 60 + 100

in 0-1 knapsack, we either use an item as a whole or do not use it



Unbounded knapsack

- Does the greedy solution works for unbounded knapsack? NO
  - A counterexample showing why greedy does not work for solving unbounded knapsack
  - There are Infinite number of all items



  - Optimal solution for this example: two of item4 and two of item2
    - Total weight=10
    - Total value = 42
  - Greedy algorithm: The fourth item has the best value/weight ratio:
    - Total weight=9 → 3x3
    - Total value=39 → 3x13

Coin Changing

Coin Change Problem

- Problem:
  - Given coin denominations: 1, 5, 10, 25, 100, design an algorithm to pay a given amount to customers using **fewest** number of coins
  - Example: 34 cents = 25 + 5 + 1 + 1 + 1 + 1
- Solution:
  - Cashier's Algorithm (A greedy algorithm):
  - At each iteration, add coin of the **largest** value that does not take us past the amount to be paid
  - Cashier's algorithm generates the optimal solution



Question

- Does the greedy algorithm always work for *any* set of coin denominations?

Coin Changing

Input: A set of coin denominations $d_1, d_2, \ldots, d_n$ and a positive integer M

Output: finding $A = [a_1, \ldots, a_n]$ such that $M = \sum_{i=1}^{n} d_i a_i$ and $\sum_{i=1}^{n} a_i$ is minimized.

Output: the minimum number of denominations (coins) that sum up to exactly M, or determine that it is impossible if no such combination exists.

- $\mathbf{a_i}$ denotes the number of coins of denomination d_i that are used
- The total value of all the chosen coins must be exactly equal to M.
  - We want to minimize the number of coins used

Coin Changing

Pseudocode:

```
1. CASHIERS-ALGORITHM (M, d1, d2, ... ,dn)
2.     SORT n coin denominations so that 0 < d1 < d2 < … < dn
3.     S = [0, …, n] # S[i] stores number of coins of type di
4.     for i = n to 1
5.         S[i] = floor (M / di)
6.         M = M - S[i] * di
7.     if M>0
8.         return “no solution.”
9.     return S
```

Observations: Properties of Optimal Solution

Properties of optimal solution for this coin denominations: { 1, 5, 10, 25, 100 }

1. Number of 1 cent coins ≤ 4:
   - Replace 5 of them by one 5 cents coin
2. Number of 5 cents coin ≤ 1:
   - Replace two of them by 10 cents coin
3. Number of 25 cents coin ≤ 3:
   - Replace four of them by a loonie
4. Number of 5 cents coins + number of 10 cents coins ≤ 2 10, 10, 5 → 25
   - Replace three 10 cents and zero 5 cents with one 25 cents and one 5 cents
   - Replace two 10 cents and one 5 cents with one 25 cents

10, 10, 10 → 25, 5

Proof of optimality for US. coin denomination

- **Theorem.** Cashier's algorithm is optimal for U.S. coins { 1, 5, 10, 25, 100 }
- **Proof:** by induction on amount **M**
  - **Base case:** M = 1, 2, 3, 4
  - **Induction hypothesis:** the greedy algorithm produces the optimal solution for values less than M
  - **Induction step:** having the induction hypothesis, show that the greedy algorithm produces the optimal solution for M
    - The greedy algorithm choses d_k such that $d_k \leq M < d_{k+1}$
    - Any optimal solution must choose d_k
      - if not, it needs to add up to M using only enough coins of type $d_1, \ldots, d_{k-1}$ to add up to M
        - However, no optimal solution can do this → against our observations
    - **Optimal and greedy choose the same for M.** Now, we need to show that the greedy algorithm produces optimal solution for **$M - d_k$** cents, which is true by induction

Proof of optimality for US. coin denomination

- Suppose: $5 \leq M < 10$
  - Suppose there is no 5 cents in the optimal solution.
  - The optimal solution contains only of 1 cent → contradiction
    - Number of 1 cent coins is at most 4
  - →There is no combination of lower denomination (1 cent coins) that add up to M and still is optimal
  - Therefore, the greedy solution contains at least one 5 cents coin. By induction the greedy solution is optimal for M-5. → The greedy solution is optimal

Properties of optimal solution for this coin denominations: { 1, 5, 10, 25, 100 }

1. Number of 1 cent coins ≤ 4
2. Number of 5 cents coin ≤ 1
3. Number of 25 cents coin ≤ 3
4. Number of 5 cents coins + number of 10 cents coins ≤ 2

Proof of optimality for US. coin denomination

- Suppose: $10 \leq M < 25$
  - Suppose there is no 10 cents in the optimal solution.
  - The optimal solution contains only of 1 cent and 5 cents coins
  - In an optimal solution, we can get at most 9 using 1 and 5 cents coins
  - →There is no combination of lower denomination (1 and 5 cents coins) that add up to M and still is optimal
  - Therefore, the greedy solution contains at least one 10 cents coin. By induction the greedy solution is optimal for M-10. → The greedy solution is optimal

Properties of optimal solution for this coin denominations: { 1, 5, 10, 25, 100 }

1. Number of 1 cent coins ≤ 4
2. Number of 5 cents coin ≤ 1
3. Number of 25 cents coin ≤ 3
4. Number of 5 cents coins + number of 10 cents coins ≤ 2

Proof of optimality for US. coin denomination

- Suppose: 25 ≤ M <100
  - Optimal solution must contain one 25 cents coin
  - Suppose there is no 25 cents in the optimal solution.
  - The optimal solution contains only of 1, 5, and 10 cents coins
  - Without a 25 cents coin, the largest amount an optimal solution could make using (10, 5, 1) is 24
    - 2 ten cents and 4 one cent
    - Example: To construct 26 using only (10, 5, 1) we get 10 + 10 + 5 + 1
      - It contradicts the last observation: Number of 5 cents coins + number of 10 cents coins should be at most 2 which is not in the above example
  - →**There is no combination of lower denomination (10, 5, 1) that add up to M and still is optimal**
  - Therefore, the greedy solution contains at least one 25 cents coin. By induction the greedy solution is optimal for M-25.
    → The greedy solution is optimal

Properties of optimal solution for this coin denominations: { 1, 5, 10, 25, 100 }

1. Number of 1 cent coins ≤ 4
2. Number of 5 cents coin ≤ 1
3. Number of 25 cents coin ≤ 3
4. Number of 5 cents coins + number of 10 cents coins ≤ 2

Proof of optimality for US. coin denomination

- Suppose: 100 ≤ M < 200
  - Optimal solution must contain one 100-cent coin
  - **There is no combination of lower denomination (25, 10, 5, 1) that add up to M and still is optimal**
  - If there are no 100-cents in optimal solution, the largest amount an optimal solution could makeis
    - 4 (one cent) + 2 (ten cents) + 3 (twenty-five cents) which add up to 99
    - → cannot make M → Optimal (and the greedy) contains a loonie .
    - By inductive hypothesis, greedy is optimal for M-100. So greedy is optimal for M
- Exercise: Suppose: 200 <= M

Properties of optimal solution for this coin denominations: { 1, 5, 10, 25, 100 }

1. Number of 1 cent coins ≤ 4
2. Number of 5 cents coin ≤ 1
3. Number of 25 cents coin ≤ 3
4. Number of 5 cents coins + number of 10 cents coins ≤ 2

Cashier’s algorithms

- Is cashier’s algorithm optimal for any set of denominations?
  - No. Consider U.S. postage: 1, 10, 21, 34, 70, 100, 350, 1225, 1500.
  - Cashier’s algorithm: 140¢ = 100 + 34 + 1 + 1 + 1 + 1 + 1 + 1.
  - Optimal: 140¢ = 70 + 70.



Conclusion: Greedy algorithms not always optimal

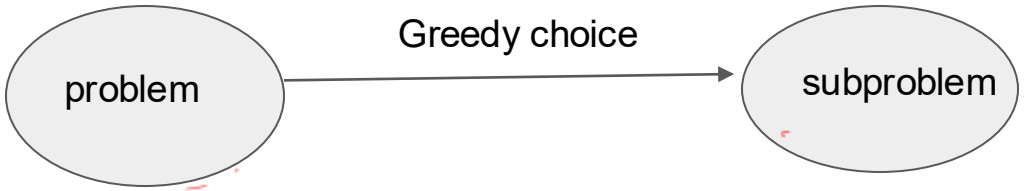
Coin changing

- The Cashier algorithm is optimal for U.S. coin denomination (old Canadian coin system)
- This algorithm does not always produce an optimal solution
  - Example:
    - US. post denomination
    - New Canadian coin system
- If the denominations include 1, then there is always a solution
- **Why the proof does not work for US. post denomination?**
  - Left as an exercise

because these denominations do not have the multiplicative relationship in Canadian old denominations.(1, 5, 10, 25, 100)

When can we use greedy technique?

- Greedy algorithm works on problems having the following properties
  - **Greedy-choice property:**
    - A locally optimal choice is globally optimal
      - Make whatever choice seems best at the moment, then solve the subproblem that remains
      - A greedy algorithm makes its choice before solving any subproblems
      - The choice does not depend on future choice or subproblem solution
      - they run extremely quickly – generally, linear or close-to-linear in the problem size.



  - **Optimal Substructure**
    - An optimal solution to the problem contains within it optimal solutions to the subproblems

When can we use greedy technique?

- Problems where greedy algorithm works
  - Activity selection/interval scheduling
  - Fractional Knapsack
  - Coin Changing
- Problems where greedy algorithm does not work
  - Integer knapsack → 0-1 knapsack
  - Unbounded knapsack

Proof techniques for greedy algorithms

Proof ideas:

- Greedy chooses the same as optimal: coin changing problem
  - Just use induction
- Stays ahead:
  - At each step, greedy makes a better choice (or not worse) than the optimal
  - Use that to prove optimality
  - Interval scheduling
- Exchange argument:
  - Transform the optimal solution to the greedy solution step by step
  - We may reach a contradiction (the optimal is not optimal anymore) which shows that the optimal solution is the same as greedy solution
  - fractional knapsack